

Le client/serveur dans le cas du Web

Olivier Flauzac & Cyril Rabat

Licence 3 Info - Info0503 - Introduction à la programmation client/serveur

2020-2021



Cours n°8

Rappel des problématiques liées au Web
Présentation d'AJaX

Version 20 octobre 2020

Table des matières

- 1 Exécutions longues et parallèles
- 2 *Javascript* et AJaX
- 3 La classe XMLHttpRequest
- 4 Exemples d'utilisation d'AJaX
- 5 AJaX : *JQuery* et *Fetch*
- 6 Le *framework* Node.js

Table des matières

1 Exécutions longues et parallèles

- Problématiques
- Quelques solutions

2 *Javascript* et *AJaX*

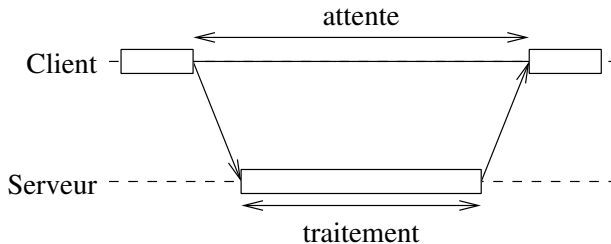
3 La classe `XMLHttpRequest`

4 Exemples d'utilisation d'*AJaX*

5 *AJaX* : *JQuery* et *Fetch*

6 Le *framework* `Node.js`

Rappel sur le modèle client/serveur



Que se passe-t-il si le traitement sur le serveur est long?

Cas d'un traitement long

- Dans le cas d'une programmation "classique" (hors Web) :
 - ↪ Dépend des technologies utilisées
- Mode non connecté ou connecté ?
 - Mode non connecté :
 - ↪ Doit-on attendre le message ?
 - ↪ Est-il perdu ?
 - Mode connecté : combien de temps doit-on attendre ?
- Suivant l'intergiciel utilisé : problèmes similaires

Remarques

- Notion de temps dans un système réparti
- Attention à la synchronisation !

Cas particulier du Web

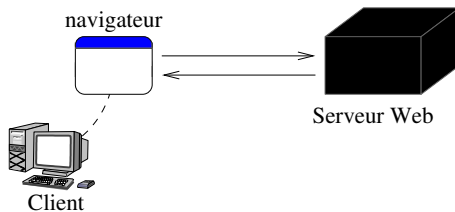
- Durée d'exécution longue :
 - Sur le client :
 - ↪ Dégradation de l'expérience utilisateur
 - ↪ Interruption par le navigateur (script ne répond pas...)
 - Sur le serveur : dépend de la configuration
 - ↪ Interruption de l'exécution du script
 - ↪ Blocage du script voire du site par l'hébergeur
- Gestion de la durée d'exécution :
 - Configuration du serveur Web
 - Configuration de PHP...

Exemples de mise en œuvre

- Boucle infinie dans un script PHP
- Requête BD lourde

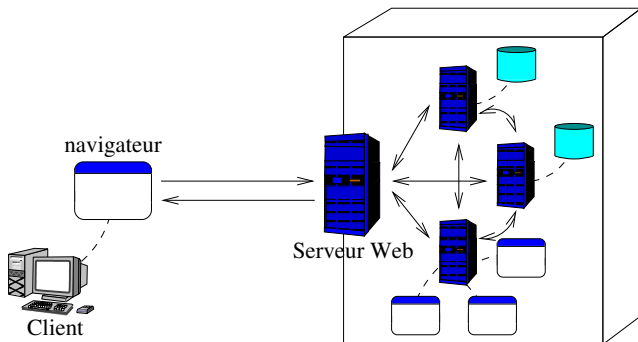
Estimation de la durée d'exécution (1/2)

Illustration de la vision Web



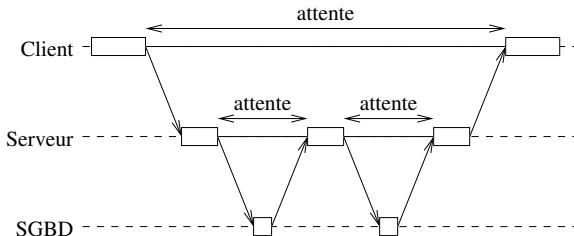
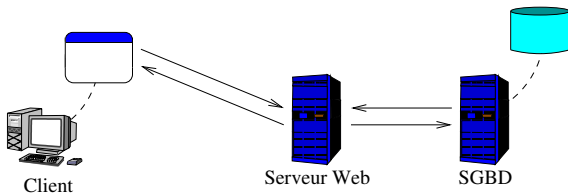
Estimation de la durée d'exécution (2/2)

Illustration de la vision Web



Quelle sera la durée du traitement de la requête du client ?

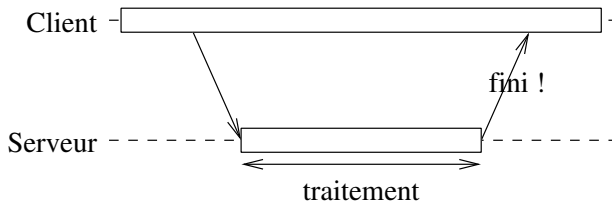
Exemple avec un SGBD



Cas d'une exécution longue

- Que se passe-t-il sur le serveur en cas d'exécution trop longue ?
 - Le processus en question est tué
 - Serveur saturé : plus de réponse pour les autres clients, etc.
- Si la configuration du serveur permet l'exécution longue :
 - Le client interrompt la connexion
 - ↪ Temps de réponse considéré comme une erreur
 - L'utilisateur interrompt la connexion
 - ↪ Gêne d'utilisation

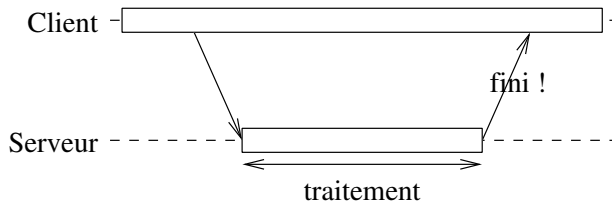
Première solution : exécution parallèle (1/2)



- Le client envoie une requête
- Il poursuit son exécution pendant le traitement
- Il intercepte les résultats



Première solution : exécution parallèle (2/2)

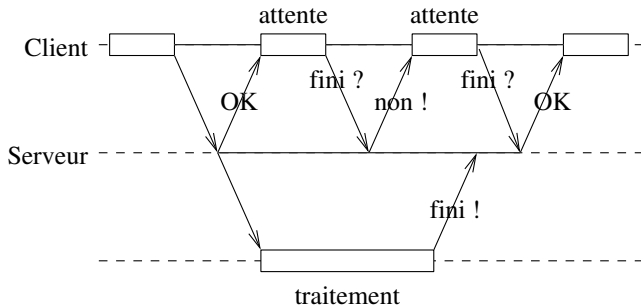


Remarques

- Appel asynchrone
- Il faut que le client puisse intercepter l'appel en cours d'exécution

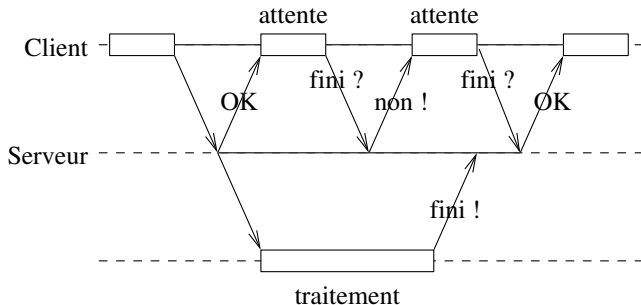


Deuxième solution : exécution parallèle avec client actif (1/2)



- Lors de la requête, traitement réalisé dans un *thread*
- Le client reçoit un accusé de réception
- À intervalles réguliers : interrogation du serveur

Deuxième solution : exécution parallèle avec client actif (2/2)



Remarques

- Ne règle pas le problème du temps maximum à attendre !
- Et que faire en attendant ?

Table des matières

1 Exécutions longues et parallèles

2 *Javascript* et AJaX

- Le Web et *Javascript*
- AJaX

3 La classe XMLHttpRequest

4 Exemples d'utilisation d'AJaX

5 AJaX : *JQuery* et *Fetch*

6 Le *framework* Node.js

Les technologies du Web

- Modèle Web :
 - Basé sur le protocole HTTP
 - Modèle client/serveur
- Différents niveaux de dynamicité :
 - Aucune : fichiers HTML+CSS et médias (photos, vidéos...)
 - Côté serveur : PHP, ASP, CGI, J2EE, *nodeJS*...
 - Côté client : principalement du code mobile
 - Code "léger" : *Javascript*, *flash*, *ActiveX*...
 - Code "lourd" : *applet*...

Rappels sur *Javascript*

- Créé en 1995, standardisé sous ECMAScript en 1997
 ↪ *Javascript* est une implémentation d'ECMAScript
- Langage de script exécuté au sein d'une page Web
 - Intégré dans le code HTML
 - Interprété par le navigateur
 ↪ Attention cependant aux incompatibilités entre navigateurs !
- Basé en partie sur la syntaxe du *Java*
- Objectif : rendre la page dynamique
- Une API complète
- Différents composants/éléments/langages liés :
 - AJaX
 - JSON

Quelques mots sur le cache

- Rappels lors de l'accès à un URL :
 - Script principal chargé par le navigateur
 - Éléments secondaires (images, scripts, etc.) : utilisation du cache
- Scripts *Javascript* situés dans des fichiers séparés :
 - ↪ Évite la surcharge du code HTML
 - ↪ Libère de la bande passante (scripts non rechargés à chaque fois)
- Problèmes dans le cas de la programmation *Javascript* :
 - ↪ Modifications des scripts non prises en compte sur le client !
- Solutions :
 - Vider l'historique du navigateur
 - Inclure le *Javascript* dans le fichier HTML
 - ↪ Uniquement en mode développement
 - Configuration du serveur Web (empêche la mise en cache)
 - Manuellement avec un numéro de version :
 - ↪ `<script src="script.js?v=<?php echo time(); ?>" ...`

AJaX

- AJaX pour **A**synchronous **J**ava**S**cript and **X**ML
- Exploiter *Javascript* pour l'envoi de requêtes HTTP
- Les réponses peuvent ensuite être analysées :
 - ↪ Généralement au format XML ou JSON
 - ↪ Peut être de tout type (exemples : JSON, HTML, etc.)
- Utilisation de la classe *Javascript* : XMLHttpRequest
- Du côté du serveur : aucun changement !

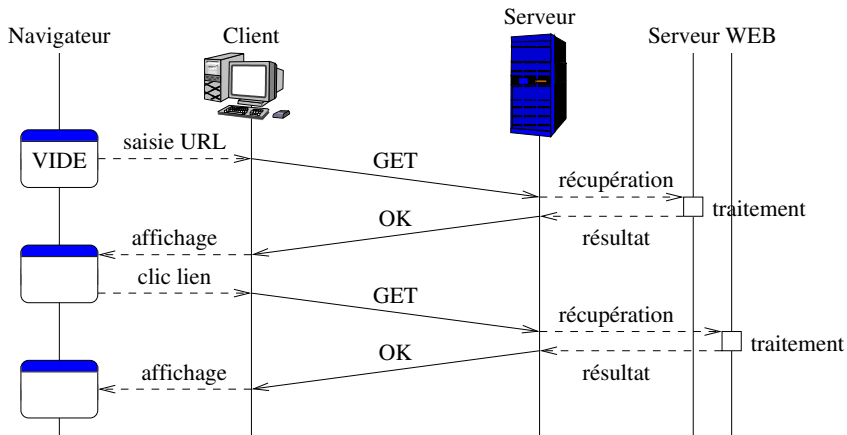
Fonctionnement d'AJaX

- Sans AJaX, à chaque requête HTTP, nouvelle page créée par le navigateur
- Avec AJaX, possibilité d'exécuter des requêtes au sein d'une même page Web :
 - ↔ Pas de rechargement de la page
- Intérêts :
 - Pages dynamiques (modification du DOM de la page)
 - Plus grande ergonomie pour l'utilisateur
 - Pas de nouveau langage (réutilisation de standards)
 - Données présentes sur le client et sur le serveur
- Les appels peuvent être asynchrones...
 - ↔ L'utilisateur garde la main pendant la réception des données
- ...ou synchrones (mais obsolètes)

Limites d'AJaX

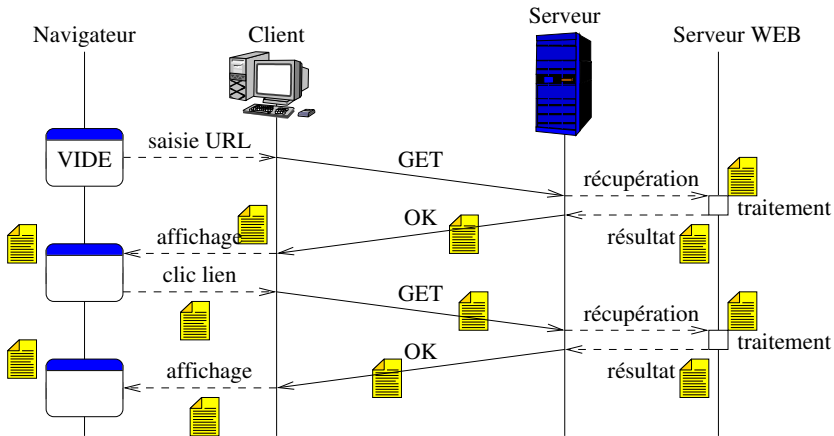
- Croisement de domaine :
 - Par mesure de sécurité, impossible de récupérer des données sur un autre domaine avec AJaX
 - Travail actuel du W3C, même si des implémentations (propriétaires) existent
- Techniquement, toutes requêtes HTTP autorisées
↔ dont GET et POST
- Attention à l'encodage des données (UTF-8)

Différences d'appels



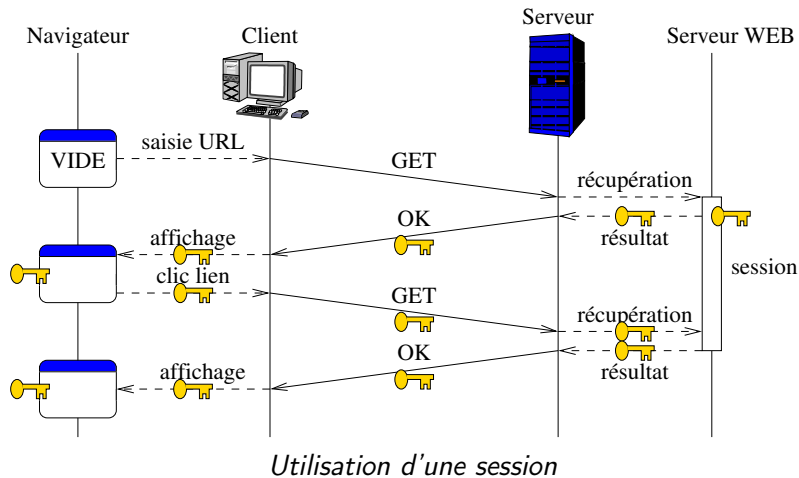
Série de requêtes/réponses HTTP classiques

Différences d'appels

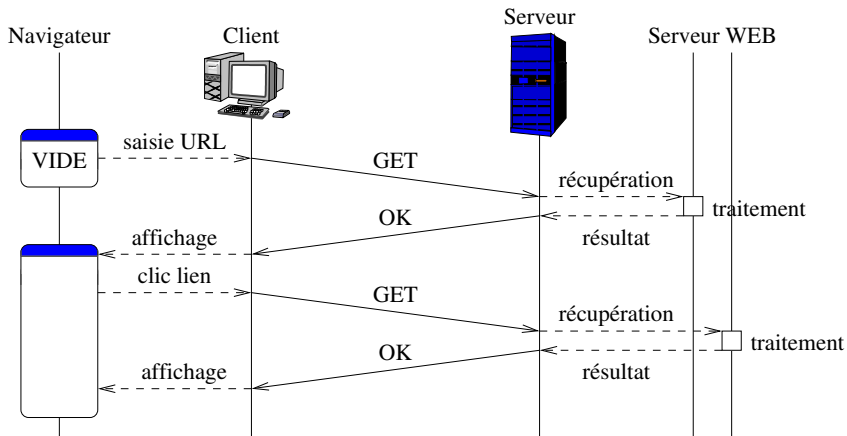


Utilisation d'un cookie

Différences d'appels



Différences d'appels



Utilisation d'AJaX (avec couplage possible avec les cookies et les sessions)

Table des matières

- 1 Exécutions longues et parallèles
- 2 *Javascript* et *AJAX*
- 3 La classe XMLHttpRequest
 - Méthodes de base
 - Un peu plus loin
- 4 Exemples d'utilisation d'AjAx
- 5 *AJAX : JQuery* et *Fetch*
- 6 Le *framework* Node.js

Présentation de la classe XMLHttpRequest

- Manipulation d'AJaX :
 - ↪ Utilisation de la classe XMLHttpRequest
- Possède un constructeur par défaut :
 - ↪ `var requeteHTTP = new XMLHttpRequest();`
- Permet l'envoi de requêtes HTTP et le traitement de la réponse
- Si plusieurs requêtes simultanées, plusieurs objets nécessaires
- Une fois créé, il faut initialiser l'objet :
 - Préparation de la requête
 - Modification de l'en-tête

Préparation de la requête et envoi

- Création de l'URL fourni à l'objet XMLHttpRequest :
 $\hookrightarrow$ `requeteHTTP.open("GET", "toto.html");`
- Le prototype de la méthode `open` :
 `open(méthode, URL, synchrone, user, password)`
 - méthode : GET, POST, HEAD, PUT, DELETE
 - URL : URL
 - synchrone (option) : `true` si appel asynchrone (par défaut)
 - user et password (option) : authentification HTTP
- Une fois l'URL spécifié, envoi de la requête :
 $\hookrightarrow$ `requeteHTTP.send();`

Appel asynchrone

- Appel à `send` non bloquant
- À la réception de la réponse : un *handler* (ou *callback*) est appelé
- Plusieurs types de *handler* suivant l'état de l'appel :
 - `onloadstart`, `onload`, `onloadend`
 - `onprogress`
 - `onabort`
 - `onerror`
 - `ontimeout`
- Pour spécifier le *handler* :
↪ `requeteHTTP.onloadend = function() { ... };`
- Il s'agit d'un pointeur de fonction : possible d'utiliser une fonction existante
↪ `requeteHTTP.onloadend = traitement;`
↪ La fonction `traitement` est définie ailleurs

Handler générique

- Un autre *handler* générique : `onreadystatechange`
- La fonction est appelée à chaque changement d'état de l'objet
- Le *handler* est appelé plusieurs fois!!!
- Comment savoir si la réponse a été reçue?
 - ↪ Vérification de l'état de l'objet

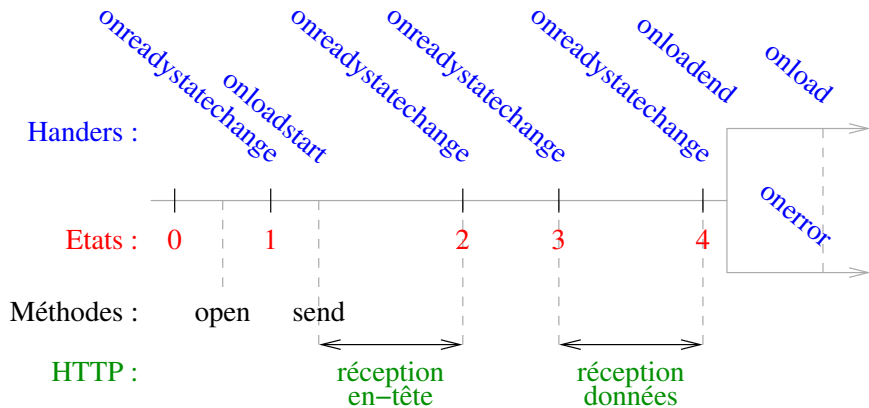
L'état de l'objet

- L'objet XMLHttpRequest prend différents états successifs
- État récupéré à l'aide de l'attribut readyState
- Les différents états :
 - UNSENT (=0) : l'objet est construit
 - OPENED (=1) : la méthode open a été correctement invoquée
 - HEADER_RECEIVED (=2) : en-tête final reçu
 - LOADING (=3) : corps en cours de réception
 - DONE (=4) : toutes les données ont été reçues

Récupération du statut HTTP

- Deux attributs :
 - `status` : code d'état HTTP (exemple : 200)
 - `statusText` : message associé (exemple : OK)
- Utilisable dès que l'en-tête est reçu
- Par défaut à 0 (ou en cas d'erreur)

Résumé d'un appel



Récupération des en-têtes

- Méthode `getResponseHeader (nomDuChamp)`
- Récupère la valeur associée à un champ
- Par exemple :
`alert (requeteHTTP.getResponseHeader ("Content-type")) ;`
- Récupération de tous les champs avec
`getAllResponseHeaders ()`

Fixer un compte-à-rebours

- Possibilité de fixer un temps maximum pour la réponse
- Utilisation de l'attribut `timeout` (en millisecondes)
- Si la réponse n'est pas reçue avant l'expiration du compte-à-rebours :
 ↪ *handler* `ontimeout` exécuté
- Autre solution pour interrompre une requête : méthode `abort()`

Attention au support des navigateurs...

Modifier l'en-tête

- Possibilité d'ajouter des champs dans l'en-tête :
↪ À faire avant l'envoi de la requête
- Utilisation de la méthode `setRequestHeader` :
↪ `setRequestHeader(nomDuChamp, valeur)`
- Si le champ existe, combinaison avec la valeur existante

Remarques sur les cookies

- La gestion des cookies est automatique en *Javascript*
↪ `document.cookie`
- Pas de modification de l'en-tête manuelle

Table des matières

- 1 Exécutions longues et parallèles
- 2 *Javascript* et *AJAX*
- 3 La classe `XMLHttpRequest`
- 4 Exemples d'utilisation d'AJaX
 - Exemple 1 : Chuck Norris facts
 - Récupération de JSON
 - Jouons avec les `cookies`
 - Encodage d'URL et POST
- 5 *AJAX : JQuery et Fetch*
- 6 Le *framework* `Node.js`

Description de l'exemple

- Page Web complétée dynamiquement :
 - Nouveaux éléments reçus via AJaX
 - Insertion dans le DOM
- Éléments utilisés :
 - Fichier HTML :
 - Contient le *Javascript* exploitant AJaX
 - Paragraphe vide complété avec la réponse AJaX
 - ↪ Identifiant *destAJaX*
 - ↪ Possible aussi de le construire à l'exécution
 - Bouton appelant la méthode qui lance la requête AJaX
 - Script PHP : génération de texte

Code Javascript

```
<html>
  <head>
    <script type="text/javascript">
      var requeteHTTP = new XMLHttpRequest();
      requeteHTTP.onload = function() {
        document.getElementById("destAJaX").innerHTML = requeteHTTP.
          responseText;
      }
      requeteHTTP.onerror = function() {
        document.getElementById("destAJaX").innerHTML = "Erreur_!";
      }
      function exemple() {
        requeteHTTP.open("GET", "generateur.php");
        requeteHTTP.send();
      }
    </script>
  </head>
  ...
```

Code PHP

```
<?php
define("CITATIONS", [
    "Chuck_Norris_peut_faire_des_ronds_avec_une_équerre.",
    "Peter_Parker_a_été_mordu_par_une_araignée,_Clark_Kent_a_été_mordu_
      _par_Chuck_Norris.",
    "Chuck_Norris_se_souvient_très_bien_de_son_futur",
    "Chuck_Norris_a_déjà_compté_jusqu'à_l'infini._Deux_fois.",
    "Google,_c'est_le_seul_endroit_où_tu_peux_taper_Chuck_Norris...",
    "Certaines_personnes_portent_un_pyjama_Superman._Superman_porte_un_
      _pyjama_Chuck_Norris.",
    "Chuck_Norris_donne_fréquemment_du_sang_à_la_Croix-Rouge._Mais_
      _jamais_le_sien.",
    "Chuck_Norris_et_Superman_ont_fait_un_bras_de_fer,_le_perdant_
      _devait_mettre_son_slip_par_dessus_son_pantalon."
]);
echo CITATIONS[rand(0, sizeof(CITATIONS) - 1)];
```

Exécution

- ❶ Création du document contenant l'élément paragraphe vide
- ❷ Lorsque le bouton est pressé, l'objet `XMLHttpRequest` est créé
- ❸ La requête est ensuite créée puis envoyée :
⇨ L'utilisateur reprend la main
- ❹ Lorsque la réponse est reçue : exécution de la fonction *handler*
⇨ Le texte est ajouté dans la balise



Exécution

- ➊ Création du document contenant l'élément paragraphe vide
- ➋ Lorsque le bouton est pressé, l'objet `XMLHttpRequest` est créé
- ➌ La requête est ensuite créée puis envoyée :
↪ L'utilisateur reprend la main
- ➍ Lorsque la réponse est reçue : exécution de la fonction *handler*
↪ Le texte est ajouté dans la balise

À noter

- Les balises HTML seront interprétées par le navigateur :
↪ Si le type de contenu est `application/html`
- Est-ce que le *Javascript* est exécuté ?



Exécution

- ❶ Création du document contenant l'élément paragraphe vide
- ❷ Lorsque le bouton est pressé, l'objet `XMLHttpRequest` est créé
- ❸ La requête est ensuite créée puis envoyée :
↪ L'utilisateur reprend la main
- ❹ Lorsque la réponse est reçue : exécution de la fonction *handler*
↪ Le texte est ajouté dans la balise

À noter

- Les balises HTML seront interprétées par le navigateur :
↪ Si le type de contenu est `application/html`
- Est-ce que le *Javascript* est exécuté? **NON!!!**
↪ C'est au programmeur de l'exécuter manuellement



Exécution d'un script *Javascript* reçu

Modification du code de l'*handler*

```
// Mise à jour du contenu
document.getElementById("destAJaX").innerHTML =
    requeteHTTP.responseText;

// Récupération des balises script
var elementsJS =
    document.getElementById('destAJaX').getElementsByTagName('script');

// Exécution des balises
for(var i in elementsJS) {
    eval(elementsJS[i].text);
}
```



Exécution d'un script *Javascript* reçu

Modification du code de l'*handler*

```
// Mise à jour du contenu
document.getElementById("destAJaX").innerHTML =
    requeteHTTP.responseText;

// Récupération des balises script
var elementsJS =
    document.getElementById('destAJaX').getElementsByTagName('script');

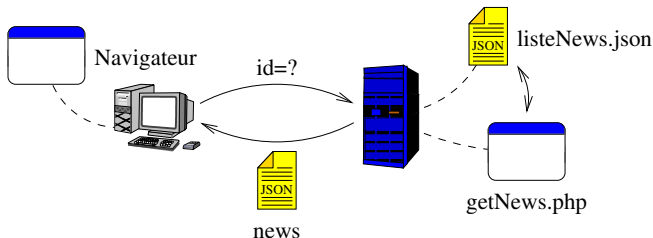
// Exécution des balises
for(var i in elementsJS) {
    eval(elementsJS[i].text);
}
```

Il n'y a pas que du *Javascript* entre les balises `<script>`



Application de *news* (1/2)

- Récupération de *news* depuis un serveur
- Toutes les *news* sont contenues dans un fichier JSON :
↪ Pour simplifier, mais elles peuvent être contenues dans une BD
- L'utilisateur peut parcourir les *news* une à une :
↪ Le numéro de la *news* désirée est envoyée (via GET)
- Le serveur envoie une *news* après l'autre



Application de *news* (2/2)

- Fichier HTML contenant le script AJaX
 - ↪ Bouton permettant d'obtenir la *news* suivante
- Script PHP qui reçoit le numéro de la *news* désirée
 - ↪ Lecture du fichier JSON
 - ↪ Vérification sur la validité du nombre
 - ↪ Envoi de la *news* + id de la suivante
- Id stocké du côté client (en *Javascript*)

Fichier index.html

```
...  
<body>  
  <h1>News</h1>  
  
  <h2 id="titreNews"></h2>  
  <p id="corpsNews"></p>  
  <p id="dateNews"></p>  
  
  <button type="button" onclick="getNews()">News suivante</button>  
</body>  
...
```

- Création de balises vides :
 ↪ Peuvent être créées à la volée

Partie Javascript (1/2)

```
var requeteHTTP = new XMLHttpRequest(); // Création de l'objet
requeteHTTP.onloadend = handler;        // Spécification de l'handler
                                        // à exécuter
var courante = 0;                      // Numéro de la news courante

// Fonction pour récupérer la news suivante
function getNews() {
    var URL = "getNews.php?id=" + courante;
    requeteHTTP.open("GET", URL, true);
    requeteHTTP.send();
}
```

- Un objet XMLHttpRequest est créé et sera réutilisé plusieurs fois :
↪ La fonction handler sera la même
- La variable courante correspond au numéro de la news courante
- La fonction getNews est appelée lorsque le bouton est cliqué

Partie Javascript (2/2)

```
function handler() {  
  if((requeteHTTP.readyState == 4) && (requeteHTTP.status == 200)) {  
    var docJSON = JSON.parse(requeteHTTP.responseText);  
  
    document.getElementById("titreNews").innerHTML = docJSON['titre'];  
    document.getElementById("corpsNews").innerHTML = docJSON['corps'];  
    document.getElementById("dateNews").innerHTML = docJSON['date'];  
    courante = docJSON['suivant'];  
  }  
  else {  
    // Gestion d'erreur  
  }  
}
```

- Les données reçues sont en JSON
 ↪ Il suffit de récupérer les valeurs

Fichier getNews.php

```
header("Content-type:_application/json");

$listeNews = json_decode(file_get_contents("listeNews.json"), true);

if(!isset($_GET['id']))
    $num = 0;
else {
    $num = intval($_GET['id']);
    if(($num < 0) || ($num >= count($listeNews->news)))
        $num = 0;
}

$news = $listeNews[$num];
$news["suivant"] = ($num + 1) % sizeof($listeNews);
echo json_encode($news);
```

Fichier listeNews.json

```
[  
  {  
    "titre" : "Titre_1",  
    "corps" : "Corps_1",  
    "date" : "01/10/2015"  
  },  
  {  
    "titre" : "Titre_2",  
    "corps" : "Corps_2",  
    "date" : "02/10/2015"  
  },  
  ...  
]
```

Positionnement des données

- Le nombre de *news* n'est pas connu par le client :
↪ Quand s'arrêter ? Comment éviter les erreurs ?
- Solution précédente :
 - Le serveur spécifie le numéro de la *news* suivante dans la réponse
 - Le client met à jour sa variable locale en fonction
- Autres solutions :
 - Utilisation d'un `cookie` :
 - Le serveur crée un `cookie` contenant le numéro de la *news* courante
 - L'identifiant est envoyé à chaque fois par le client
 - Le serveur peut le mettre à jour
 - Utilisation d'une session :
↪ Le numéro de la *news* courante est sauvé sur le serveur

Description de l'application

Fonctionnement

- Compteur stocké sur le client
- Envoi de la valeur sur le serveur

Scripts

- Création d'un `cookie` par le script PHP
- Dans le `cookie` : déclaration d'un compteur `id`
- Récupération/suppression avec *Javascript*
- Avec AJaX : envoi au serveur et récupération du résultat

Javascript : partie AJaX (1/2)

```
var requeteHTTP = new XMLHttpRequest();

requeteHTTP.onloadend = function() {
    if(requeteHTTP.status == 200)
        document.getElementById("destination").innerHTML =
            requeteHTTP.responseText;
};

function update() {
    requeteHTTP.open("GET", "script.php");
    requeteHTTP.send();
}
```

Javascript : récupération du cookie (2/2)

```
function reset() {  
    var expiration = new Date();  
    expiration.setTime(today.getTime() - 1);  
    document.cookie = "id=0;expires=" + expiration.toGMTString();  
    document.getElementById("destination").innerHTML =  
        "Réinitialisation";  
}  
  
function recuperationValeur() {  
    var expression = new RegExp("(?:;|_)?id=([^;]*)?");  
  
    if (expression.test(document.cookie))  
        document.getElementById("destination").innerHTML =  
            "Valeur_actuelle_=" + decodeURIComponent(RegExp["$1"]);  
    else  
        document.getElementById("destination").innerHTML =  
            "Pas_de_valeur_fixée.";  
}
```

Fichier HTML

```
<body onload="javascript:recuperationValeur();">
  <h1>Test avec un cookie</h1>

  <div id="destination"></div>

  <button type="button" onclick="javascript:update();">
    Envoi requête
  </button>
  <button type="button" onclick="javascript:reset();">
    Réinitialisation
  </button>
</body>
```

Script PHP

```
<?php
// Récupération de la valeur du cookie
if(isset($_COOKIE['id'])) {
    $id = $_COOKIE['id'];
    $id++;
}
else
    $id = 1;

// Modification de sa valeur
setcookie("id", $id, 0);

echo "Vous_avez_réalisé_$id_fois_l'appel";
```

Encodage d'URL et POST

Données envoyées en GET

- Les données sont codées dans l'URL :
 - ↪ Ajout de "?" puis des variables séparées par "&"
 - ↪ Chaque variable : `nomVariable=valeur`
- Utilisation de la fonction *Javascript* `encodeURIComponent`

Données envoyées en POST

- Comment envoyer des données avec POST ?
 - ↪ Les données sont spécifiées comme avec GET
- Modifier l'en-tête avec le `Content-Type` suivant :
 - ↪ `application/x-www-form-urlencoded`

Table des matières

- 1 Exécutions longues et parallèles
- 2 *Javascript* et AJaX
- 3 La classe XMLHttpRequest
- 4 Exemples d'utilisation d'AJaX
- 5 AJaX : *JQuery* et *Fetch*
 - Utilisation d'AJaX avec *JQuery*
 - Utilisation d'AJaX avec *Fetch*
- 6 Le *framework* Node.js

JQuery

- Bibliothèque *Javascript*
- Permet :
 - D'enrichir les possibilités
 - Simplifier le développement
 - Système de sélecteurs puissant
 - Ajouter des effets
 - etc.
- À inclure comme un script *Javascript* :
 ↪ Utiliser le CDN

AJaX avec JQuery

- Solution permettant de simplifier les appels AJaX
- Fonction `ajax` qui prend en paramètre :
 - Type de requête HTTP (GET, POST, etc.)
 - L'URL
 - Les données
 - ↪ Format standard (`"nom=" + nom + "&prenom=" + prenom`)
 - ↪ JSON (`{ 'nom' : nom, 'prenom' : prenom }`)
 - Les *handlers* pour la réussite, l'échec
- Permet aussi simplement de modifier l'en-tête :
 - Type des données envoyées
(`application/x-www-form-urlencoded`)
 - Type des données à recevoir
 - etc.
- Objet `jqXHR` retourné

Code générique d'une requête AJaX

```
requete = $.ajax(  
  {  
    type: 'POST',  
    url: '',  
    data: '',  
    dataType: 'json'  
  }  
);  
requete.done(  
  function (reponse, texteStatut, jqXHR) {  
    // Requête réussie  
  }  
);  
requete.fail(  
  function (jqXHR, texteStatut, erreurLevee){  
    // Erreur !!!  
  }  
);
```

Exemple : les familles des séries télévisées

- Application permettant de récupérer les personnages de séries
- Constituée des fichiers suivants :
 - `index.html` : formulaire pour la saisie
 - `monfichier.js` : script *Javascript* avec la requête AJaX
 ↪ Interagit avec `donnees.php`
 - `series.json` : données brutes
 - `donnees.php` : retourne les données en fonction des saisies de l'utilisateur
- Fonctionnement :
 - L'utilisateur saisit le nom de la série et éventuellement, le nom de la famille
 - La liste des familles ou des membres est affichée

Extrait du fichier index.html

```
...  
<label ref="serie">Série</label>  
<input type="text" id="serie"/>  
<label ref="famille">Famille</label>  
<input type="text" id="famille"/>  
  
<button type="button" onclick="recherche()">  
  Recherche  
</button>  
  
<div id="erreur"></div>  
<div id="resultat"></div>  
  
<script src="https://code.jquery.com/jquery-3.5.1.min.js"></script>  
<script src="script.js" type="text/javascript"></script>  
...
```

Le script PHP `donnees.php`

- Récupère le nom de la série et éventuellement, le nom de famille
↪ En POST
- Chargement des données depuis le JSON
- Retourne un document JSON contenant :
 - Le code de retour (OK ou KO)
 - La liste des données (séries, familles, personnages)

Extrait du fichier *Javascript* script.js (1/2)

```
function recherche() {  
    requete = $.ajax({  
        type: 'POST',  
        url: 'donnees.php',  
        data: { "serie" : $('#serie').val(),  
                "famille" : $('#famille').val() },  
        dataType: 'json'  
    });  
    requete.fail(function (jqXHR, textStatus, errorThrown){  
        $('#erreur').text("Problème_de_requête_AJaX.").show();  
        console.log(jqXHR);  
    });  
    ...  
}
```

Extrait du fichier *Javascript* script.js (2/2)

```
...
requete.done(function (response, textStatus, jqXHR) {
    if("erreur" in response)
        $('#erreur').text(response['erreur']).show();
    else
        $('#erreur').hide();

    $('#resultat').text(response['titre']).append('<ul>').show();
    for(i = 0; i < response['donnees'].length; i++)
        $('#resultat').append("<li>" + response['donnees'][i] + "
            </li>");
    $('#resultat').append("</ul>");
});
}
```

Fetch

- Alternative à XMLHttpRequest
- Définition générique des objets Request et Response
- Plus de fonctionnalités
- Apporte de la souplesse et de la puissance
- Pas besoin d'inclure JQuery

Attention

L'API est récente et de nombreuses fonctionnalités ne sont pas supportées par tous les navigateurs

Requête AJaX

```
function recherche() {  
    var serie = document.getElementById('serie').value;  
    var famille = document.getElementById('famille').value;  
    fetch('donnees.php', {  
        method: "POST",  
        body: 'serie=' + encodeURIComponent(serie) +  
              '&famille=' + encodeURIComponent(famille),  
        headers: {  
            'Accept' : 'application/json',  
            'Content-Type': "application/x-www-form-urlencoded"  
        }  
    }).then(response => {  
        if(response.ok) {  
            response.json().then(analyse).catch(error => {  
                console.error(error);  
            });  
        } else  
            console.error(response.status);  
    }).catch(error => { console.error(error); });  
}  
...
```

Traitement de la réponse

```
function analyse(response) {  
    var resultat = document.getElementById('resultat');  
    var erreur = document.getElementById('erreur')  
  
    if("erreur" in response) {  
        erreur.innerHTML = response['erreur'];  
        erreur.style.display = "block";  
    }  
    else  
        erreur.style.display = "none";  
  
    resultat.style.display = "block";  
    resultat.innerHTML = response['titre'] + "<ul>";  
    for(i = 0; i < response['donnees'].length; i++)  
        resultat.innerHTML += "<li>" + response['donnees'][i] +  
                               "</li>";  
    resultat.innerHTML += "</ul>";  
}
```

Table des matières

- 1 Exécutions longues et parallèles
- 2 *Javascript* et *AJaX*
- 3 La classe `XMLHttpRequest`
- 4 Exemples d'utilisation d'*AJaX*
- 5 *AJaX* : *JQuery* et *Fetch*
- 6 Le *framework* Node.js
 - Présentation
 - Retour sur l'exemple des séries

Qu'est-ce que *Node.js* ?

- Plateforme logicielle libre en *Javascript* :
↪ Open source, gratuite
- Basée sur le moteur de Chrome Javascript V8
- Permet de développer des serveurs Http
- Très largement utilisé pour des applications commerciales :
↪ *PayPal, Groupon, LinkedIn, Microsoft, Yahoo !, etc.*
- Basé sur *Javascript* mais avec des fonctionnalités supplémentaires
↪ L'accès aux fichiers, par exemple

Comment l'utiliser en INFO0503 ?

- Permet de remplacer les serveurs en *Java* ou PHP
- Utilisation basique :
 - Création d'un serveur Http simple
 - Récupération de données et génération de JSON
- Utilisation du framework *Express*
 - ↪ Permet d'accélérer le développement d'applications

Application de base

```
var http = require("http");

// Démarrage du serveur Http
http.createServer(function (request, response) {
  response.writeHead(200, {'Content-Type': 'text/plain'});
  response.end('Hello_World\n');
}).listen(8080);

// Affichage des info
console.log('Serveur_démarré_http://127.0.0.1:8080/');
```

- Démarrage : `node main.js`

Fichier principal

```
// Chargement du module http
const http = require('http');

// Récupération de l'application (fichier app.js)
const app = require('./app');

// Configuration de l'application
const hostname = '127.0.0.1';
const port = 3000;
app.set('port', port);

// Démarrage du serveur
const server = http.createServer(app);

server.listen(port, hostname, () => {
  console.log(`Serveur prêt http://${hostname}:${port}/`);
});
```

Utilisation du framework *express*

```
const express = require('express');
const app = express();

const bodyParser = require('body-parser');
const fs = require("fs");

// Pour parser les données en POST
app.use(bodyParser.urlencoded({ extended: true }));

// Pour récupérer les éléments statiques (répertoire public)
app.use(express.static('public'));

// Handler sur l'URL 'donnees'
app.post("/donnees", function(request, response) {

})

module.exports = app;
```

Retour sur les séries

```
// Envoi les séries ou les familles
function envoiData(data, cle, propriete, response, titre, message) {
  // Récupération des données depuis le JSON
  series = [];
  for(i = 0; i < data[cle].length; i++)
    series.push(data[cle][i][propriete]);

  // Envoi des données
  response.setHeader('Content-Type', 'application/json');
  response.end(JSON.stringify({
    erreur: message,
    titre: titre,
    donnees: series
  })));
}

// Envoi des membres d'une famille
function envoiMembres(data, response, message) {
  // Idem
}
```

Le *handler* principal (1/2)

```
app.post("/donnees", function(request, response) {
  fs.readFile("series.json", "utf8", function(error, data) {
    data = JSON.parse(data);

    if(("serie" in request.body) && (request.body['serie'] != "")) {
      i = 0;
      while((i < data['series'].length) &&
        (data['series'][i]['intitulé'] != request.body['serie']))
        i++;

      if(i >= data['series'].length)
        envoiData(data, "series", "intitulé", response,
          "Liste_des_séries_disponibles",
          "Cette_série_est_inconnue");
    }
    else {
      if(("famille" in request.body) && (request.body['famille']
        != "")) {
        j = 0;
        while((j < data['series'][i]['familles'].length) &&
          (data['series'][i]['familles'][j]['nom'] != request.
            body['famille']))
          j++;
      }
    }
  });
});
```

Le *handler* principal (2/2)

```
        if(j >= data['series'][i]['familles'].length)
            envoiData(data['series'][i], "familles", "nom", response
                , "Liste_des_familles_disponibles", "Cette_famille_
                  est_inconnue");
        else
            envoiMembres(data['series'][i]['familles'][j], response,
                "");
    }
    else
        envoiData(data['series'][i], "familles", "nom", response,
            "Liste_des_familles_disponibles",
            "La_famille_n'a_pas_été_spécifiée");
    }
}
else
    envoiData(data, "series", "intitulé", response,
        "Liste_des_séries_disponibles",
        "La_série_n'a_pas_été_spécifiée");
});
});
```